

GUIA COMPLETA

Actualizacion, arquitectura y operacion del sistema QA 1.4.9

Asociacion Comunal

Estado de la entrega

API 1.4.9 desplegada, base disponible, paquete Windows 1.4.9 publicado y release de GitHub creado. El acceso publico por ngrok esta temporalmente bloqueado por limite mensual de ancho de banda.

Commit final de develop: **0b52cfb242897491f150883a83f74a9987def3d5**

Ultima integracion de codigo: PR #24 de **Gerson Bermudez**, autor de la arquitectura unificada por capas.

Fecha de corte: 20 de agosto de 2026 - zona America/El_Salvador

Como usar esta guia

Esta guia sirve como documento de arquitectura, manual de operacion y procedimiento de entrega. Los comandos usan valores de ejemplo y nunca incluyen tokens, contraseñas ni claves privadas.

Seccion	Uso principal
1-4	Entender estado, arquitectura, historial Git y servidor.
5-8	Entender base de datos, API, clientes y actualizaciones.
9-11	Ejecutar una actualizacion completa y saber revertirla.
12-14	Validar seguridad, resolver fallos y consultar el registro 1.4.9.

Reglas operativas esenciales

- Respaldar la base antes de cualquier cambio de esquema.
- Construir desde un commit identificado y con pruebas aprobadas.
- No copiar un JAR sobre el archivo que esta ejecutando Java.
- Separar JWT_SECRET del API_SHARED_SECRET.
- No ejecutar schema.sql sobre una base existente; usar migraciones versionadas.
- Validar salud interna y externa antes de anunciar una version.
- Conservar una ruta de reversa hasta finalizar la ventana de observacion.

1. Estado final de QA 1.4.9

Componente	Estado	Evidencia
Codigo	Integrado	PR #23 primero; PR #24 de Gerson fusionado al final.
API	Activa	Release 1.4.9-api-0b52cfb; /health devuelve base available.
Base	Preparada	Tablas y columnas de V002-V005 ya existentes en QA.
Windows	Publicado	Manifest version 1.4.9 y SHA-256 verificado.
GitHub	Publicado	Release qa-v1.4.9 sobre el commit final.
Tunel publico	Bloqueado	ngrok ERR_NGROK_725 por cuota mensual agotada.

Interpretacion correcta

La entrega esta instalada en el servidor. La aplicacion no podra descubrirla desde Internet mientras la URL publica de ngrok siga rechazando trafico. No es un fallo del manifest 1.4.9 ni de la API.

Artefactos Windows

Archivo	Tamano	SHA-256
AsociacionComunalQA-1.4.9-win64.zip	69,978,173 bytes	1fcb1324...5268fcda
AsociacionComunalQA-1.4.9-delta.zip	13,778,546 bytes	2afbb890...281f64c
update-manifest.json	19 archivos	Referencia el hash del delta

2. Arquitectura del sistema

El backend sigue la propuesta de Gerson: un solo modulo Maven genera un fat JAR y organiza responsabilidades en capas. El frontend no importa entidades del servidor.



Capas del fat JAR

Capa	Responsabilidad	Ejemplos
controller	Traduce HTTP, estado y JSON.	AuthController, ViviendaController
service	Reglas de negocio y orquestacion.	MiembroService, UpdateService
dao	SQL y mapeo persistente.	UsuarioDAO, ViviendaDAO
domain	Entidades y DTO de entrada/salida.	Miembro, LoginResponse
config	Entorno, puerto y conexion.	AppConfig, DBConnection
middleware	Autenticacion transversal.	JwtAuthMiddleware, AuthMiddleware
util	Funciones compartidas.	PasswordHasher, DateUtils

Credito

El PR #24 permanece asociado a Gerson Bermudez y fue la ultima integracion de codigo antes de la entrega 1.4.9.

3. Ramas Git e historial de integracion

Orden	PR / rama	Resultado
1	PR #23 - feature/dui-temporary-password	Miembros, documentos, viviendas, Android y proteccion del workflow.
2	PR #24 - chore/fix-backend-architecture-to-web-service	Arquitectura de Gerson, adaptada y fusionada al final.
Final	develop @ 0b52cfb	Fuente unica de la entrega QA 1.4.9.

Comprobaciones recomendadas

```
git fetch origin
git status --short --branch
git log --graph --decorate --oneline -15 origin/develop
git show --stat 0b52cfb242897491f150883a83f74a9987def3d5
gh pr view 24 --repo Miascn/AsociacionComunal
```

No publicar desde una rama ambigua

El versionado debe partir de develop y de un SHA registrado. Un nombre de version sin SHA no permite reproducir ni auditar la entrega.

4. Como esta hecho el servidor QA

El servidor Linux ejecuta servicios systemd de usuario. La API escucha solamente en 127.0.0.1:8080; ngrok publica el acceso HTTPS. MySQL tambien se consume localmente.

Ruta / servicio	Funcion
~/config/asociacion-comunal/api.env	Variables API, JWT y MySQL; permisos 600.
~/config/systemd/user/asociacion-api.service	Arranca Java 21 con el JAR de current.
~/config/systemd/user/asociacion-ngrok.service	Mantiene el tunel HTTPS.
~/apps/asociacion-api/releases/	Versiones inmutables del JAR.
~/apps/asociacion-api/current	Enlace simbolico a la version activa.
~/apps/asociacion-api/updates	ZIP, delta, manifiesto y version Windows.
~/apps/asociacion-api/android-updates	APK, version, notas y checksum Android.
~/backups/asociacion-comunal	RespalDOS previos a entregas.

Variables requeridas

```
API_PORT=8080
API_SHARED_SECRET=<secreto tecnico de 64 caracteres>
JWT_SECRET=<secreto JWT diferente, de 64 caracteres>
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=asociacion_comunal
DB_USER=<usuario de aplicacion>
DB_PASSWORD=<contrasena segura>
AUTH_ACCESS_TTL_MINUTES=15
AUTH_REFRESH_TTL_DAYS=30
```

Nunca imprimir secretos

Validar presencia y longitud, no el valor. El token tecnico viaja en el paquete QA actual, por lo que debe limitarse a actualizaciones y rotarse si se filtra.

5. Base de datos y migraciones

La base operativa es asociacion_comunal. El usuario asociacion_api posee SELECT, INSERT, UPDATE y DELETE, pero no permisos DDL. Las migraciones deben ejecutarse con un administrador autorizado.

Migracion	Cambio	Dependencia
V002	Crea sesion_usuario para access y refresh token.	usuario
V003	Agrega requiere_cambio_clave.	usuario
V004	Tipos de documento y pais de origen.	miembro
V005	Vivienda, residentes y relaciones.	miembro

Respaldo previo

```
backup="$HOME/backups/asociacion-comunal/pre-entrega.sql.gz"
export MYSQL_PWD="$DB_PASSWORD"
mysqldump -h "$DB_HOST" -P "$DB_PORT" -u "$DB_USER" \
  --single-transaction --skip-lock-tables --skip-triggers \
  --no-tablespaces "$DB_NAME" | gzip -9 > "$backup"
gzip -t "$backup"
sha256sum "$backup"
```

Validacion previa al DDL

- Consultar information_schema para confirmar tipos INT e INT UNSIGNED de las claves foraneas.
- Comprobar si cada tabla, columna y constraint ya existe.
- Aplicar una migracion una sola vez y registrar fecha, ambiente y operador.
- No usar schema.sql sobre QA: es un bootstrap para instalaciones nuevas.
- Probar /health y consultas de negocio despues del cambio.

QA 1.4.9

V002, V003, V004 y V005 ya estaban presentes. Se creo respaldo y no se repitio DDL.

6. API, rutas y autenticacion

Canal	Rutas	Autenticacion
Salud	/health	Publica; informa servicio y base.
JavaFX admin	/api/admin/auth/login	Credenciales MySQL -> JWT.
Modulos	/api/miembros, /proyectos, /viviendas	Bearer JWT.
Usuarios	/api/usuarios/*	JWT y rol ADMIN o ADMINISTRADOR.
Android sesion	/api/auth/* y /api/me	Access/refresh opacos almacenados como hash.
Windows publish	/api/updates/*	API_SHARED_SECRET.
Android download	/api/mobile/updates/*	Publica para el instalador.
Android publish	/api/updates/android/publish	API_SHARED_SECRET.

Compatibilidad de contraseñas

- Formato de Gerson: salt hexadecimal, dos puntos, hash hexadecimal; PBKDF2 SHA-256, 120000 iteraciones.
- Formato de sesiones Android: pbkdf2_sha256\$iteraciones\$saltBase64\$hashBase64.
- El verificador 1.4.9 acepta ambos formatos para no bloquear cuentas existentes.
- Las contraseñas nuevas de usuarios administrativos siguen la logica de Gerson.

Pruebas minimas de proteccion

Solicitud	Esperado
GET /health	200
POST admin login con credenciales falsas	401
POST Android login con credenciales falsas	401
GET /api/miembros sin JWT	401
GET /api/updates/windows/manifest sin secreto	401
GET /api/mobile/updates/android/manifest	200 o 404 si no hay APK

7. Como se conectan las aplicaciones

JavaFX Windows

- Lee config/qa.properties incluido en la imagen de aplicacion.
- api.url debe ser HTTPS; api.token es el secreto tecnico para el canal de actualizaciones.
- El login envia nombreUsuario y clave a /api/admin/auth/login.
- SessionManager conserva el JWT solo durante la sesion de la aplicacion.
- MiembroApiClient, ProyectoApiClient y ViviendaApiClient envian Bearer JWT.
- UpdateService consulta el manifest con el token tecnico y verifica SHA-256 antes de instalar.

Android

- Envia username y password a /api/auth/login.
- Recibe accessToken, refreshToken, expiresIn y user.
- Guarda tokens mediante SessionStore y cifrado local.
- Rota el refresh token y revoca sesiones al cerrar sesion.
- Consulta /api/me para reconstruir identidad y miembro.
- El actualizador consulta /api/mobile/updates/android/manifest y valida el APK.

Separacion deliberada

JavaFX usa JWT segun la arquitectura de Gerson. Android conserva el contrato de sesiones opacas para no romper instalaciones existentes.

8. Versionado y paquetes de actualizacion

Artefacto	Consumidor	Contenido
win64.zip	Primera instalacion / soporte	Imagen completa jpackage con runtime.
delta.zip	Actualizador instalado	Archivos app modificados y launcher.
update-manifest.json	Actualizador	Version, hash, tamano y hashes por archivo.
version.txt	Clientes antiguos	Version semantica exacta.
sha256.txt	Paquete completo	SHA-256 del ZIP win64.

Reglas de version

- Usar X.Y.Z, por ejemplo 1.4.9.
- La version del manifest debe coincidir con version.txt.
- El servidor rechaza una version inferior a la publicada.
- El hash del ZIP completo debe coincidir con sha256.txt.
- El hash y tamano del delta deben coincidir con update-manifest.json.
- No publicar automaticamente en cada push: solo workflow_dispatch con publish=true.

Construccion reproducible

```
mvn clean test
mvn package -DskipTests
./scripts/build-qa-package.ps1 -Version 1.4.9

# Resultados en dist/
# AsociacionComunalQA-1.4.9-win64.zip
# AsociacionComunalQA-1.4.9-delta.zip
# update-manifest.json, version.txt, sha256.txt
```

9. Procedimiento completo de actualizacion

Fase A - Preparacion

- Confirmar alcance, version y commit.
- Revisar ramas, PR, cambios de base y compatibilidad de clientes.
- Ejecutar pruebas API y frontend.
- Confirmar que no hay secretos ni credenciales predeterminadas en Git.
- Asegurar que el PR que debe recibir credito sea la ultima integracion.

Fase B - Base de datos

- Crear respaldo comprimido y verificarlo con gzip -t y SHA-256.
- Consultar information_schema.
- Aplicar solo migraciones ausentes con cuenta DDL autorizada.
- Comprobar tablas, columnas, constraints y permisos.

Fase C - API

```
release_id="1.4.9-api-<sha-corto>"
app_root="$HOME/apps/asociacion-api"
release_dir="$app_root/releases/$release_id"

install -d -m 700 "$release_dir"
install -m 500 "$HOME/asociacion-api.jar.upload" \
"$release_dir/asociacion-api.jar"
readlink -f "$app_root/current" > "$app_root/previous.txt"
ln -sf "$release_dir" "$app_root/current"
systemctl --user restart asociacion-api.service
curl --fail http://127.0.0.1:8080/health
```

Fase D - Windows

- Validar hashes localmente.
- Subir cinco artefactos a un directorio temporal del servidor.
- Publicar por POST /api/updates/windows/publish usando el secreto tecnico.
- Comprobar version.txt, sha256.txt y manifest desde localhost.
- Crear el release qa-vX.Y.Z en GitHub con ZIP completo y checksum.

10. Validacion posterior

Servidor

```
systemctl --user is-active asociacion-api.service
systemctl --user is-active asociacion-ngrok.service
readlink -f "$HOME/apps/asociacion-api/current"
curl --fail --silent http://127.0.0.1:8080/health
journalctl --user -u asociacion-api.service -n 100 --no-pager
```

Version publicada

```
cat "$HOME/apps/asociacion-api/updates/version.txt"
cat "$HOME/apps/asociacion-api/updates/sha256.txt"
sha256sum "$HOME/apps/asociacion-api/updates/AsociacionComunalQA-win64.zip"
```

Cliente QA

- Abrir una copia 1.4.6 y esperar la deteccion de 1.4.9.
- Aceptar, observar descarga, validacion de hash, instalacion y reinicio.
- Confirmar version 1.4.9 en la interfaz.
- Cerrar y abrir: no debe ofrecer la misma version.
- Probar login administrativo, miembros, viviendas y proyectos.
- Probar Android login, refresh, logout y /api/me.

Criterio de exito

No basta con que el ZIP exista: salud, autenticacion, manifest, descarga y reinicio deben verificarse de extremo a extremo.

11. Reversion y recuperacion

Revertir API

```
app_root="$HOME/apps/asociacion-api"
IFS= read -r previous < "$app_root/previous.txt"
ln -sf "$previous" "$app_root/current"
systemctl --user restart asociacion-api.service
curl --fail http://127.0.0.1:8080/health
```

Revertir paquete Windows

- Conservar los archivos estables de la version anterior antes de activar una nueva.
- Restaurar ZIP estable, delta, manifest, version y checksum como una unidad.
- No mezclar version.txt de una entrega con ZIP de otra.
- Una version inferior puede requerir procedimiento de rollback explicito, no publicacion normal.

Restaurar base

- Detener escrituras y confirmar el punto de restauracion.
- Guardar otro respaldo del estado fallido para investigacion.
- Restaurar con cuenta administrativa en una ventana controlada.
- Reconciliar el JAR con el esquema restaurado.
- Validar integridad y reabrir trafico solo al final.

Precaucion

No revertir automaticamente el JAR si una migracion incompatible ya transformo datos. La reversion debe considerar codigo y base como una sola unidad.

12. Seguridad y mantenimiento

Control	Practica
Secretos	64 caracteres o mas; JWT y token tecnico separados; permisos 600.
MySQL	Usuario de aplicacion sin ALTER, CREATE, DROP ni GRANT.
Red	API en loopback; solo tunel HTTPS expuesto.
Usuarios	CRUD limitado a administrador; clave hash nunca sale por JSON.
Sesiones	Tokens opacos almacenados como SHA-256; refresh rotado.
Actualizaciones	Checksum obligatorio antes de activar archivos.
Backups	Permisos 600/700, hash verificado y retencion definida.
Logs	No registrar Authorization, passwords ni archivos de configuracion.

Rutina sugerida

- Diario: salud API, servicio ngrok, espacio y errores recientes.
- Semanal: respaldo de base y prueba de lectura del respaldo.
- Mensual: rotacion de logs, revision de cuota del tunel y dependencias.
- Por entrega: respaldo, pruebas, SHA, release inmutable y checklist firmado.
- Trimestral: restauracion de prueba en ambiente aislado.

13. Diagnostico de problemas

Sintoma	Causa probable	Accion
Solo detecta 1.4.6	Manifest remoto sigue en 1.4.6 o URL publica no llega.	Consultar version.txt interno y luego URL HTTPS.
ERR_NGROK_725	Cuota mensual agotada.	Esperar reinicio de cuota, ampliar plan o migrar tunel.
/health interno 200, externo falla	Tunel, DNS o cuota.	Revisar asociacion-ngrok.service y panel del proveedor.
JavaFX 401	JWT ausente/expirado o login en ruta incorrecta.	Reiniciar sesion y usar /api/admin/auth/login.
Android 401	Sesion expirada o migracion V002 ausente.	Rotar refresh o autenticar otra vez.
API no inicia	Secreto, DB o JAR invalido.	journalctl, validar env y revertir current.
Publicacion 400	Version o checksum no coincide.	Recalcular hashes y no editar artefactos despues.
Workflow falla al descargar base	curl conserva codigo no cero en fallback.	Construir manualmente o corregir step antes de proxima entrega.

Incidente actual

QA 1.4.9 esta activo internamente. Para que 1.4.6 lo detecte, primero debe resolverse ERR_NGROK_725. Reiniciar el servicio no elimina una cuota agotada en la cuenta.

14. Registro real de la entrega 1.4.9

Hito	Resultado
PR #23	Fusionado en develop antes del PR de Gerson.
PR #24	Fusionado como ultima integracion; autor Gerson Bermudez.
Pruebas	19 API + 4 frontend aprobadas; empaquetado Maven aprobado.
Backup	20260820T1850Z-pre-1.4.9.sql.gz, gzip y SHA verificados.
Migraciones	V002-V005 detectadas; no se repitio DDL.
JWT	JWT_SECRET independiente generado en el servidor.
API	Release 1.4.9-api-0b52cfb activo; salud 200.
Proteccion	Login falso 401; miembros sin JWT 401; updates sin secreto 401.
Windows	1.4.9 publicada por API local con hashes correctos.
GitHub	qa-v1.4.9 creado sobre 0b52cfb.
Pendiente externo	Restablecer o reemplazar ngrok por ERR_NGROK_725.

Checklist de cierre

- [OK] Commit y PR final identificados.
- [OK] Pruebas y empaquetado aprobados.
- [OK] Respaldo creado y verificado.
- [OK] API desplegada con rollback disponible.
- [OK] Version 1.4.9 y checksums publicados.
- [OK] Release GitHub creado.
- [PENDIENTE] Recuperar conectividad publica del tunel.
- [PENDIENTE] Probar actualizacion real 1.4.6 -> 1.4.9 desde un equipo QA cuando vuelva el tunel.

Fin de la guia

Conservar este documento junto con el SHA del commit, el hash del respaldo y el enlace del release. Actualizarlo en cada entrega que cambie arquitectura, rutas o infraestructura.